DO NOW — RECALL

Lesson 114 · Compilers, Interpreters and Assemblers

Name: _____ Class: _____ Date: _____

1. Does free-of-charge imply that software is open source?

2. A full adder receives A=1, B=1 and carry-in=0. What are sum and carry-out?

3. For the binary value 1010, what is its decimal value?

4. A linked-list node has data and what other essential field?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

No. Price is not the same as licence/source rights.

2. Model answer

Sum 0 and carry-out 1.

3. Model answer

10.

4. Model answer

A link/reference to the next node (for a singly linked list).

By the end, you can…

• Translates a complete source unit to target code before execution, often enabling optimisation.

• Executes by translating/evaluating during running; behaviour varies by implementation.

• Converts symbolic assembly mnemonics into machine instructions.

Vocabulary — know these terms

compiler — Translates source to target code before execution.

interpreter — Executes by translating/evaluating at runtime.

assembler — Turns assembly into machine/object code.

object code — Compiled code awaiting linking or execution.

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Work through the method

Use the middle stage of the visual to follow the process and state each intermediate result.

Shared worked answer

An assembler converts the mnemonic plus operand into its encoded machine instruction. In standard LMC, LDA 20 encodes as 520. This is not an interpreter translating a high-level language.

IMPROVE: Explain one correction to your own reasoning.

NEW LEARNING

Read, annotate and explain

Compiler

Translates a complete source unit to target code before execution, often enabling optimisation.

Interpreter

Executes by translating/evaluating during running; behaviour varies by implementation.

Assembler

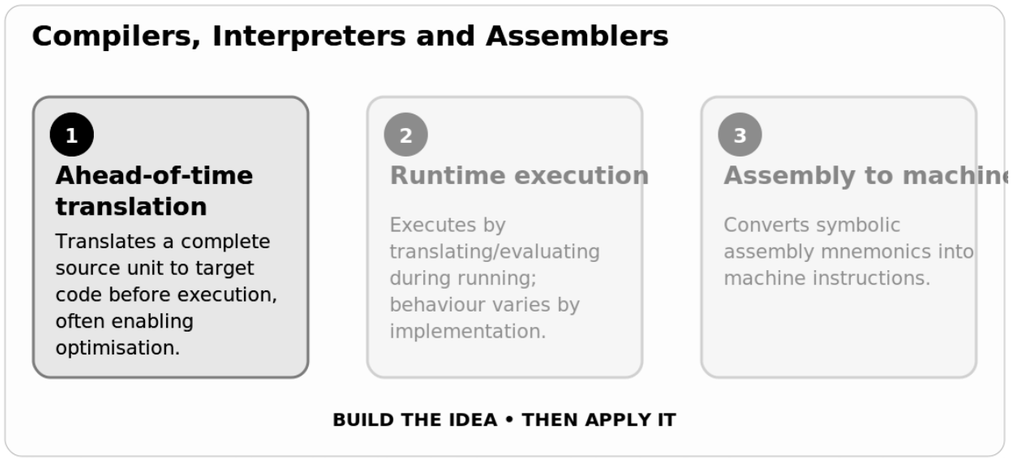
Converts symbolic assembly mnemonics into machine instructions.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

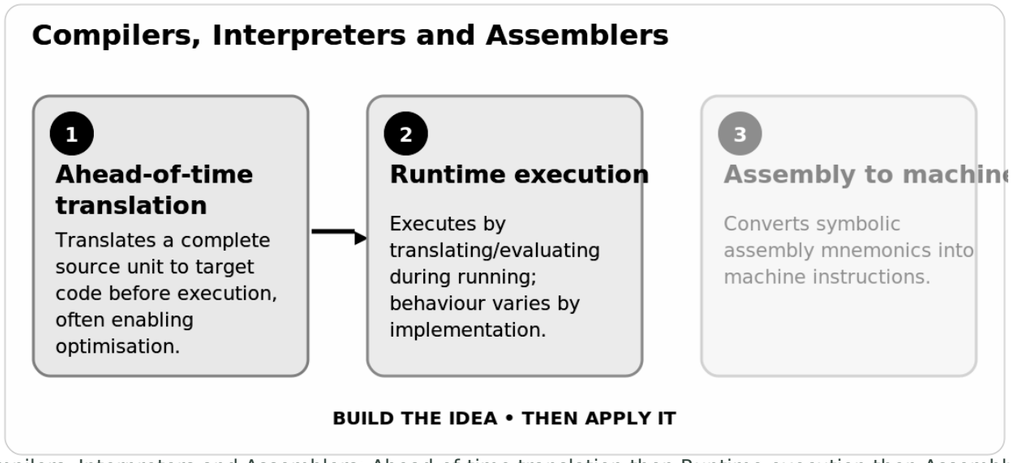
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Compilers, Interpreters and Assemblers: Ahead-of-time translation then Runtime execution then Assembly to machine

New Learning



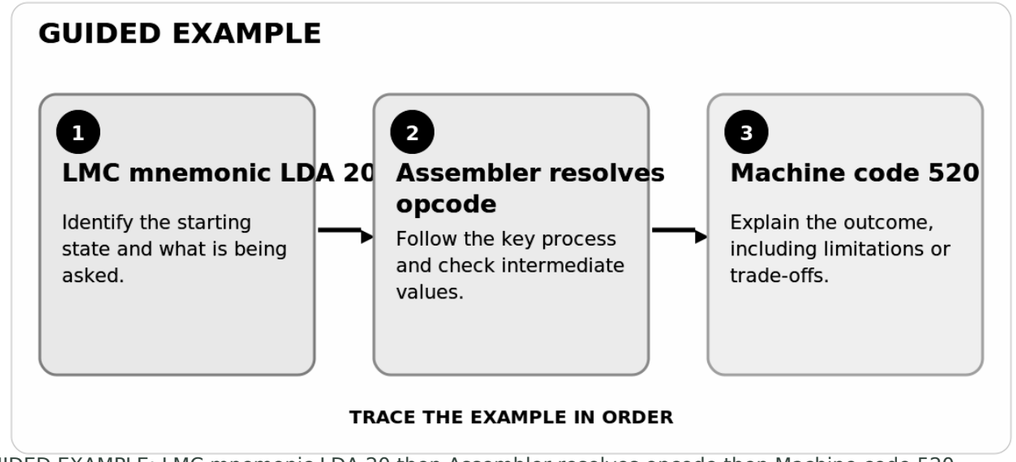
Compilers, Interpreters and Assemblers: Ahead-of-time translation then Runtime execution then Assembly to machine

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A technician reads LMC 'LDA 20' and wants the CPU's numeric instruction. What translation tool is used?

Identify the inputs, relevant facts and exact output or judgement required.



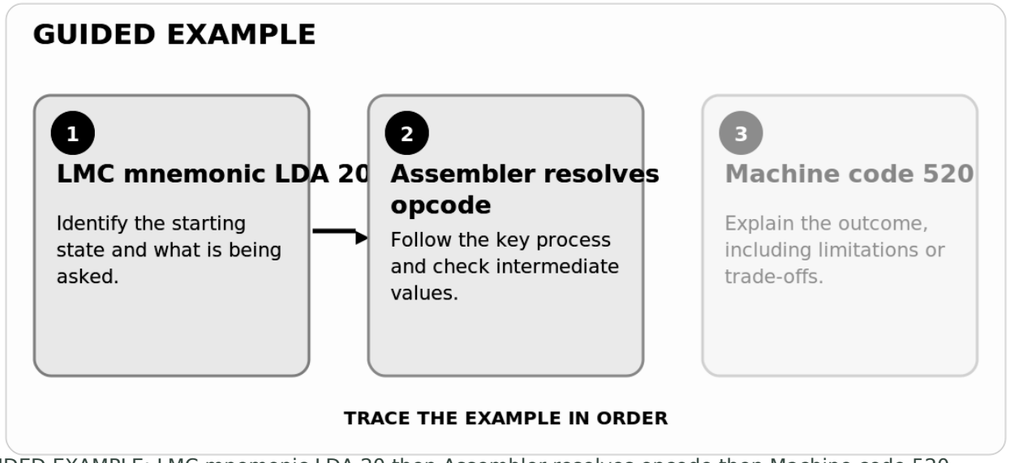
GUIDED EXAMPLE: LMC mnemonic LDA 20 then Assembler resolves opcode then Machine code 520

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A technician reads LMC 'LDA 20' and wants the CPU's numeric instruction. What translation tool is used?

Identify the inputs, relevant facts and exact output or judgement required.

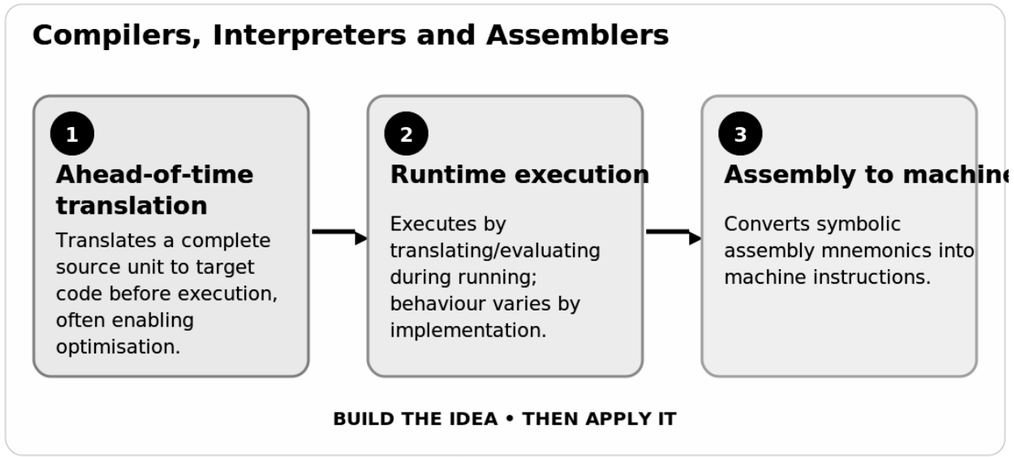


GUIDED EXAMPLE: LMC mnemonic LDA 20 then Assembler resolves opcode then Machine code 520

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Compilers, Interpreters and Assemblers: Ahead-of-time translation then Runtime execution then Assembly to machine

I DO — WORKED EXAMPLE

Follow the worked method

A team distributes an executable without its original C source. Explain the compiler's role.

Model answer

A compiler translates source to target machine/object code; linked executable can run without the source being present. Compilation can report syntax errors and apply optimisation before execution.

Check: link each claim to the scenario and use precise OCR terminology.

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

An assembler translates high-level Python syntax into binary machine code.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

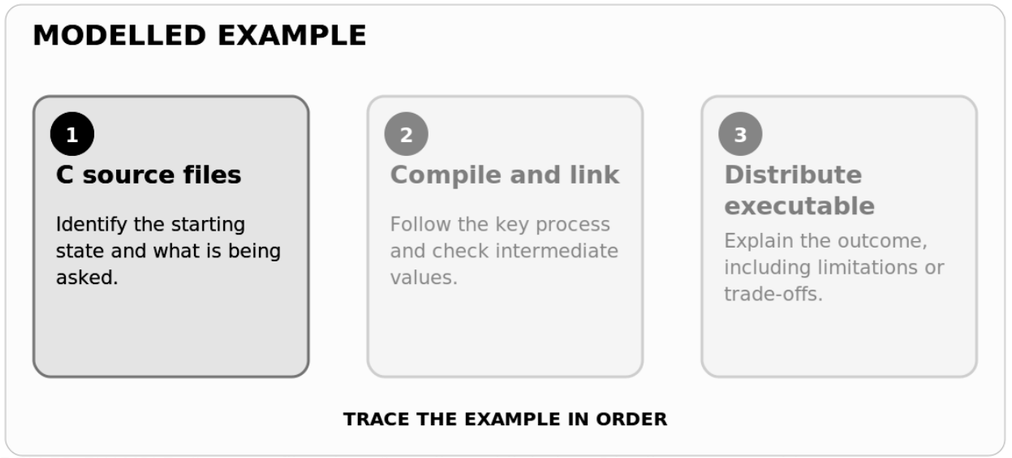
A Python-like runtime, C compiler and LMC assembler are used in one project. Compare debugging, portability and execution trade-offs. [8]

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A team distributes an executable without its original C source. Explain the compiler's role.

A compiler translates source to target machine/object code; linked executable can run without the source being present. Compilation can report syntax errors and apply optimisation before execution.



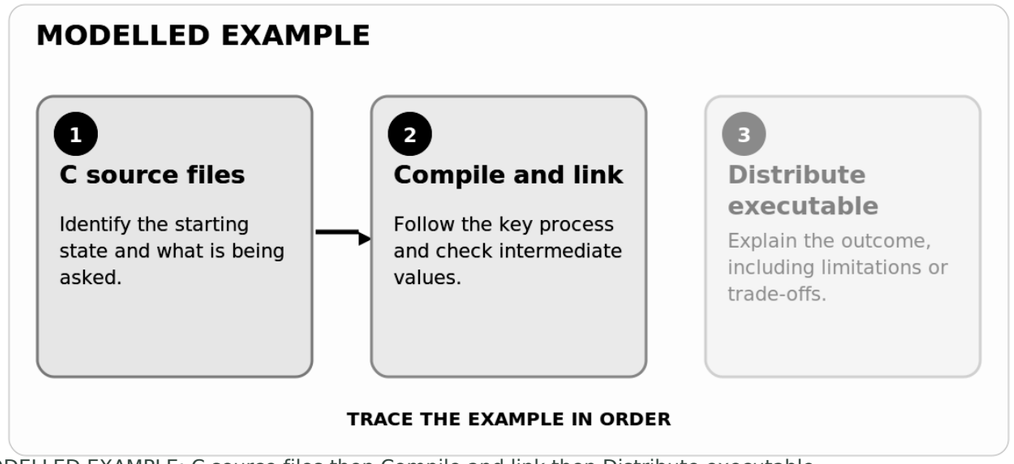
MODELLED EXAMPLE: C source files then Compile and link then Distribute executable

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A team distributes an executable without its original C source. Explain the compiler's role.

A compiler translates source to target machine/object code; linked executable can run without the source being present. Compilation can report syntax errors and apply optimisation before execution.



MODELLED EXAMPLE: C source files then Compile and link then Distribute executable

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Distinguish compiler, interpreter and assembler with a relevant input for each. [6]

Question 2

Explain two advantages of compiling a program before distribution. [6]

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: What must we decide? Identify the inputs, relevant facts and exact output or judgement required.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

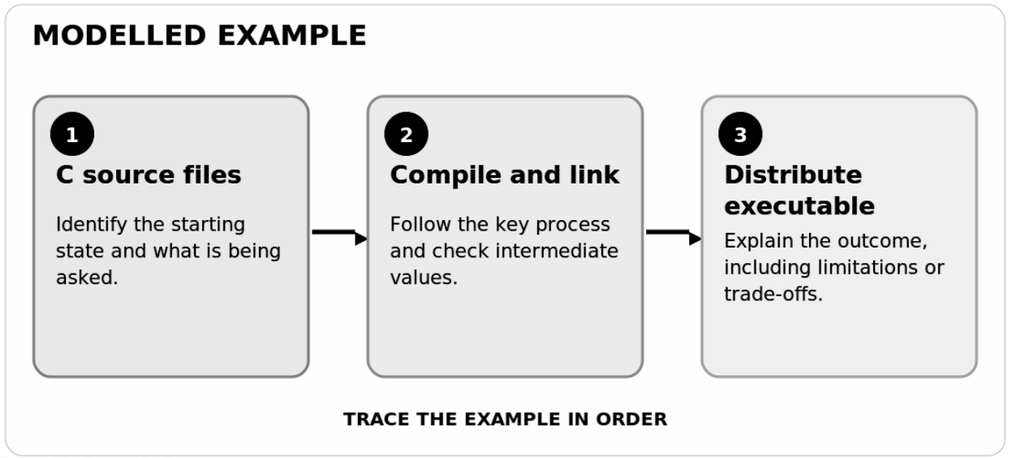
IMPROVE — Write one precise correction or stronger explanation in your own answer.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A team distributes an executable without its original C source. Explain the compiler's role.

A compiler translates source to target machine/object code; linked executable can run without the source being present. Compilation can report syntax errors and apply optimisation before execution.



MODELLED EXAMPLE: C source files then Compile and link then Distribute executable

WE DO — GUIDED PRACTICE

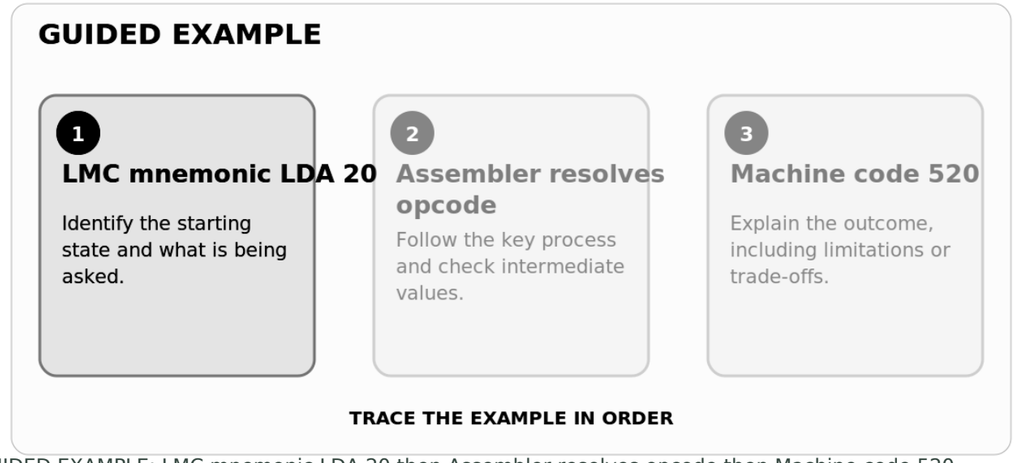
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A technician reads LMC 'LDA 20' and wants the CPU's numeric instruction. What translation tool is used?

Identify the inputs, relevant facts and exact output or judgement required.



GUIDED EXAMPLE: LMC mnemonic LDA 20 then Assembler resolves opcode then Machine code 520